gitcurious@terminal: ~/cheatsheets

SSH cheat sheet

// how ssh works, actually

15 commands for Mac & Linux (OpenSSH 10.2), flags decoded. Swap you and example.com for y

Connect

```
$ ssh you@example.com
```

Log in to a server and get its terminal.

- `you` your username on the server
- `example.com` the server's address (or IP)

```
$ ssh -p 2222 you@example.com
```

Log in to a server that isn't on the default port 22.

- `-p` port number

```
$ ssh -v you@example.com
```

Log in and print debug messages for each stage. Use it when a connection fails.

- `-v` verbose; -vv or -vvv for more detail

Keys

```
$ ssh-keygen -t ed25519 -C
  "you@example.com"
```

Create a key pair: a private key (keep it) and a .pub public key (share it).

- `-t` key type; ed25519 is the modern default
- `-C` a label, so you know which key is which

```
$ ssh-copy-id you@example.com
```

Put your public key on the server, so you log in with your key instead of your password (if the server allows it).

```
$ ssh-add ~/.ssh/id_ed25519
```

Add your key to the running ssh-agent, so you don't retype its passphrase for every connection.

```
$ chmod 700 ~/.ssh
```

Lock your SSH folder so only you can open it.

- `chmod` change mode: sets who can read/write/open a file
- `700` you: everything. everyone else: nothing

```
$ chmod 600 ~/.ssh/id_ed25519
```

Lock your private key so only you can read it. SSH ignores keys others can read.

- `600` you: read + write. everyone else: nothing

Trust

```
$ ssh-keygen -lf
  /etc/ssh/ssh_host_ed25519_key.pub
```

On the server itself (its console, or a session you already trust): print its fingerprint. It should match what SSH shows you.

- `-l` show the fingerprint
- `-f` of this file

```
$ ssh-keygen -R example.com
```

Forget a server's saved ID. Only after you've confirmed the new fingerprint is real, e.g. after a rebuild.

- `-R` remove this host from ~/.ssh/known_hosts

Move files

```
$ scp notes.txt you@example.com:~/
```

Copy one file from your computer to the server.

- `notes.txt` the file on your computer
- `you@example.com` log in as you on the server example.com (same as ssh)
- `:` separates the server from the path on it
- `~/` the folder to copy into; ~ = your home folder

```
$ rsync -avz site/
  you@example.com:~/site/
```

Copy a folder to the server. Run it again and it skips files whose size and date haven't changed. Needs rsync on both machines.

- `-a` archive: keep subfolders, permissions, dates
- `-v` verbose: list transferred files
- `-z` compress while sending
- `site/` the folder on your computer; trailing / = copy what's inside, not the folder itself
- `you@example.com` log in as you on the server example.com
- `:~/site/` after the colon: the destination folder on the server

Tunnels

```
$ ssh -L 8080:localhost:80
  you@example.com
```

Open a private web page on the server at localhost:8080 on your computer.

- `-L` local forward: your-port:target:target-port
- `localhost` as seen from the server, i.e. the server itself

```
$ ssh -D 1080 you@example.com
```

Create a SOCKS proxy on your computer. Point your browser at localhost:1080 (with proxy DNS on) and its traffic exits from the server.

- `-D` dynamic forward: the proxy listens on your port 1080

```
$ ssh -J bastion.example.com
  you@internal
```

Reach a server you can't touch directly by hopping through one you can.

- `-J` jump host: the server in the middle

~/.ssh/config shortcut

```
Host web
  HostName example.com
  User you
  IdentityFile ~/.ssh/id_ed25519
# now just:  ssh web
```

Rules

→ Your private key never leaves your machine. Share the .pub only.

→ First connection? Compare the fingerprint with a copy you trust (from the server's console or admin) before typing yes.

→ Lost a laptop? Remove its public key from ~/.ssh/authorized_keys on your servers.

@gitcurious · Save this. What should I explain next?